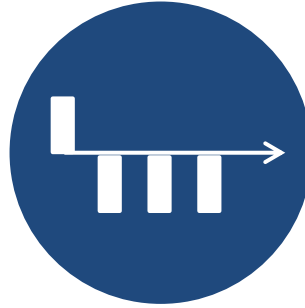




Tema 10. Comunicación serial Microcontrolador PIC18f4550



MODULO DE COMUNICACIÓN SERIE DEL PIC18F4550

2

Modulo SCI (Serial Communication interface)

- EUSART (Enhanced universal synchronous asynchronous receiver transmitter)
 - Modo Asíncrono (Full Dúplex)
 - Modo síncrono (Half Dúplex)

Modulo MSSP (Master Synchronous Serial Port)

- SPI (Serial Peripheral Interface)
- I2C (Inter-Integrated Circuit)
 - Master mode
 - Multi-Master mode
 - Slave mode

Modulo USB (universal serial bus)

CONCEPTOS SOBRE COMUNICACIÓN SERIAL

- **Comunicación serie:** Los datos se envían bit a bit por una misma línea y durante un tiempo fijo.
- **La velocidad de transmisión:** Es el numero de bit enviados por segundos (baudios).
- **Transferencia Síncrona:** Se envía una señal de reloj para sincronizar cada bit.
- **Transferencia Asíncrona:** No se envía la señal de reloj. Se necesitan reloj en el emisor y receptor a la misma frecuencia.
- **Codificación de bit NRZ (Non Return to Zero):**
 - Nivel alto: 1 (5V)
 - Nivel bajo: 0 (0V)

bit de inicio	8 o 9 bit de datos	bit de parada
---------------	--------------------	---------------

- **Transferencia síncrona:**
 - **Dispositivo maestro:** El que genera la señal de reloj y el que tiene capacidad de iniciar o finalizar una transferencia.
 - **Dispositivo esclavo:** recibe la señal de reloj no tiene la capacidad de iniciar una transferencia.
- **Transferencia Asíncrona:**
 - Dispositivo emisor y receptor
 - Se emplea reloj de igual frecuencia (velocidad de transmisión) en ambos dispositivos.
 - Se utiliza un bit de arranque (nivel bajo) y de parada (nivel alto).
 - La línea de dato inactiva permanece a nivel alto.

USART (Universal Synchronous Asynchronous Receiver Transmitter)

- Es un periférico de comunicación serie de entrada/salida.
- Conocido como Interfaz de comunicación serie (Serial Communications Interface - SCI).
- Contiene todos los generadores de señales de reloj, registros de desplazamiento y búfers de datos necesarios para realizar transmisión de datos serie de entrada/salida, independientemente de la ejecución de programa del dispositivo.
- Se puede configurarse como un sistema asíncrono dúplex completo (Full-duplex)
- También se puede configurar como un sistema síncrono semidúplex (Half Duplex)
- Permite comunicarse con dispositivos periféricos, como circuitos integrados A/D o D/A, EEPROM seriales, etc.

El EUSART integrado en el PIC18F4550 posee las siguientes características:

- **Transmisión y recepción asíncrona en modo Full-dúplex.**
- Comunicación Half-Duplex en modo síncrono.
- Caracteres de anchura de 8 – 9 bits programables.
- Detección de dirección en modo de 9 bits.
- Detección de errores por saturación del búfer de entrada.

EUSART (Enhanced Universal Synchronous Asynchronous Receiver Transmitter)

5

20.0 ENHANCED UNIVERSAL SYNCHRONOUS ASYNCHRONOUS RECEIVER TRANSMITTER (EUSART)

The Enhanced Universal Synchronous Asynchronous Receiver Transmitter (EUSART) module is one of the two serial I/O modules. (Generically, the USART is also known as a Serial Communications Interface or SCI.) The EUSART can be configured as a full-duplex asynchronous system that can communicate with peripheral devices, such as CRT terminals and personal computers. It can also be configured as a half-duplex synchronous system that can communicate with peripheral devices, such as A/D or D/A integrated circuits, serial EEPROMs, etc.

The Enhanced USART module implements additional features, including automatic baud rate detection and calibration, automatic wake-up on Sync Break reception and 12-bit Break character transmit. These make it ideally suited for use in Local Interconnect Network bus (LIN bus) systems.

The EUSART can be configured in the following modes:

- Asynchronous (full-duplex) with:
 - Auto-wake-up on Break signal
 - Auto-baud calibration
 - 12-bit Break character transmission
- Synchronous – Master (half-duplex) with selectable clock polarity
- Synchronous – Slave (half-duplex) with selectable clock polarity

The pins of the Enhanced USART are multiplexed with PORTC. In order to configure RC6/TX/CK and RC7/RX/DT/SDO as an EUSART:

- SPEN bit (RCSTA<7>) must be set (= 1)
- TRISC<7> bit must be set (= 1)
- TRISC<6> bit must be set (= 1)

Note: The EUSART control will automatically reconfigure the pin from input to output as needed.

The operation of the Enhanced USART module is controlled through three registers:

- Transmit Status and Control (TXSTA)
- Receive Status and Control (RCSTA)
- Baud Rate Control (BAUDCON)

These are detailed on the following pages in Register 20-1, Register 20-2 and Register 20-3, respectively.

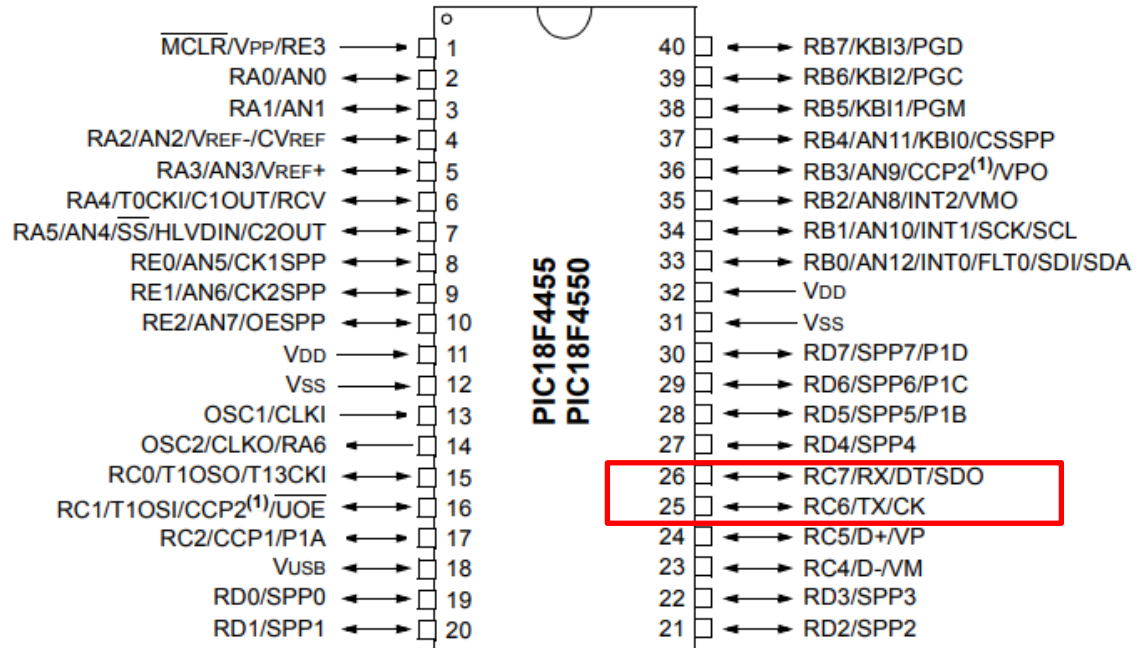
EUSART – Pines

En modo Asíncrono Full-dúplex:

- RC7/RX (Entrada de recepción)
- RC6/TX (Salida de transmisión)

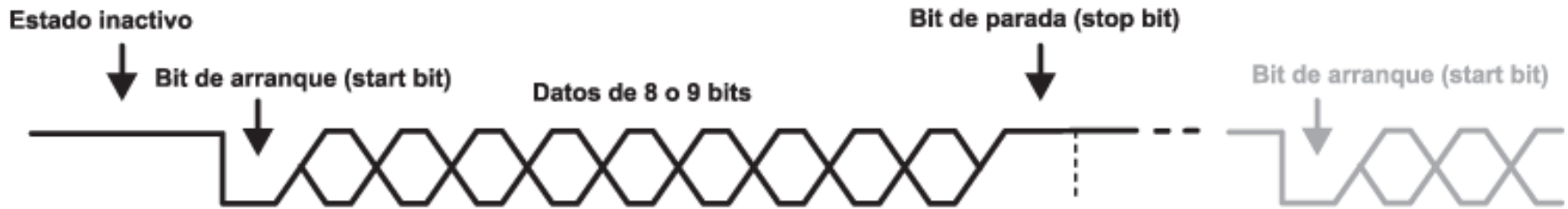
En modo Síncrono Half-Duplex:

- RC7/DT (Datos)
- RC6/CK (Reloj)



EUSART - Trama de una transmisión asíncrona

7



EUSART - Registros asociados:

- Registro de control y estado
 - TXSTA
 - RCSTA
- Registro de relación de baudio
 - SPBRG (para 8bit)
 - SPBRGH:SPBRG (para 16bit)
 - BAUDCON (control y estado)
- Registro del dato de transmisión y recepción
 - TXREG
 - RCREG



EUSART - Registros asociados:

9

TABLE 20-2: REGISTERS ASSOCIATED WITH BAUD RATE GENERATOR

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset Values on page
TXSTA	CSRC	TX9	TXEN	SYNC	SEnDB	BRGH	TRMT	TX9D	55
RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D	55
BAUDCON	ABDOVF	RCIDL	RXDTP	TXCKP	BRG16	—	WUE	ABDEN	55
SPBRGH	EUSART Baud Rate Generator Register High Byte								55
SPBRG	EUSART Baud Rate Generator Register Low Byte								55

Legend: — = unimplemented, read as '0'. Shaded cells are not used by the BRG.

EUSART – Registro TXSTA: TRANSMIT STATUS AND CONTROL REGISTER

10

CSRC	TX9	TXEN	SYNC	SENDB	BRGH	TRMT	TX9D
------	-----	------	------	-------	------	------	------

bit 7 CSRC: Clock Source Select bit

Asynchronous mode: Don't care

bit 6 TX9: 9-bit Transmit Enable bit

1 = Selects 9-bit transmission

0 = Selects 8-bit transmission

bit 5 TXEN: Transmit Enable bit

1 = Transmit enabled

0 = Transmit disabled

bit 4 SYNC: USART Mode Select bit

1 = Synchronous mode

0 = Asynchronous mode

bit 3 SENDB: Send Break Character bit

Asynchronous mode:

1 = Send Sync Break on next transmission (cleared by hardware upon completion)

0 = Sync Break transmission completed

bit 2 BRGH: High Baud Rate Select bit

Asynchronous mode:

1 = High speed

0 = Low speed

bit 1 TRMT: Transmit Shift Register Status bit

1 = TSR empty

0 = TSR full

bit 0 TX9D: 9th bit of Transmit Data, can be parity bit

-Break Character:

Es una condición especial en la línea de transmisión serial donde se mantiene el nivel lógico bajo (0) por más tiempo del que dura un carácter normal. Se usa comúnmente para:

- Sincronización entre dispositivos.
- Reinicio o señal de interrupción en algunos protocolos.
- Detección de errores o eventos especiales.

EUSART – Registro **RCSTA**: RECEIVE STATUS AND CONTROL REGISTER

11

SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
------	-----	------	------	-------	------	------	------

bit 7 SPEN: Serial Port Enable bit

1 = Serial port enabled (configures RC7/RX/DT and RC6/TX/CK pins as serial port pins)

0 = Serial port disabled

bit 6 RX9: 9-bit Receive Enable bit

1 = Selects 9-bit reception

0 = Selects 8-bit reception

bit 5 SREN: Single Receive Enable bit

Asynchronous mode: Don't care

bit 4 CREN: Continuous Receive Enable bit

Asynchronous mode:

1 = Enables receiver

0 = Disables receiver

bit 3 ADDEN: Address Detect Enable bit

Asynchronous mode 9-bit (RX9 = 1):

1 = Enables address detection, enables interrupt and loads the receive buffer when RSR<8> is set

0 = Disables address detection, all bytes are received and ninth bit can be used as parity bit

Asynchronous mode 8-bit (RX9 = 0):

Don't care.

bit 2 FERR: Framing Error bit

1 = Framing error (can be updated by reading RCREG register and receive next valid byte)

0 = No framing error

bit 1 OERR: Overrun Error bit

1 = Overrun error (can be cleared by clearing bit CREN)

0 = No overrun error

bit 0 RX9D: 9th bit of Received Data (can be parity bit, but must be calculated by user firmware)

-Address Detect Enable bit: Se usa en sistemas multipunto, donde varios dispositivos comparten la misma línea de comunicación.

- En aplicaciones donde hay varios esclavos conectados a un maestro por USART.
- El maestro envía un byte con el noveno bit en 1 para indicar que es una dirección.
- Solo el esclavo cuyo ID coincide con el byte de dirección responde y procesa los siguientes datos.

EUSART – Registro BAUDCON: BAUD RATE CONTROL REGISTER

12

ABDOVF	RCIDL	RXDTP	TXDTP	BRG16	-	WUE	ABDEN
--------	-------	-------	-------	-------	---	-----	-------

bit 7 ABDOVF: Auto-Baud Acquisition Rollover Status bit

1 = A BRG rollover has occurred during Auto-Baud Rate Detect mode (must be cleared in software)

0 = No BRG rollover has occurred

bit 6 RCIDL: Receive Operation Idle Status bit

1 = Receive operation is Idle

0 = Receive operation is active

bit 5 RXDTP: Received Data Polarity Select bit

Asynchronous mode:

1 = RX data is inverted

0 = RX data received is not inverted

Synchronous modes:

1 = Received Data (DT) is inverted. Idle state is a low level.

0 = No inversion of Data (DT). Idle state is a high level.

bit 4 TXCKP: Clock and Data Polarity Select bit

Asynchronous mode:

1 = TX data is inverted

0 = TX data is not inverted

Synchronous modes:

1 = Clock (CK) is inverted. Idle state is a high level.

0 = No inversion of Clock (CK). Idle state is a low level.

Auto-Baud Acquisition Rollover Status: permite al microcontrolador ajustar automáticamente la velocidad de comunicación serial.

- Se activa la detección automática de baudios con el bit ABDEN = 1 en el mismo registro BAUDCON.
- El microcontrolador espera recibir un carácter de sincronización para calcular la velocidad de baudios.

EUSART – Registro **BAUDCON**: BAUD RATE CONTROL REGISTER

13

ABDOVF	RCIDL	RXDTP	RXDTP	BRG16	-	WUE	ABDEN
--------	-------	-------	-------	-------	---	-----	-------

bit 3 BRG16: 16-Bit Baud Rate Register Enable bit

1 = 16-bit Baud Rate Generator – SPBRGH and SPBRG

0 = 8-bit Baud Rate Generator – SPBRG only (Compatible mode), SPBRGH value ignored

bit 2 Unimplemented: Read as '0'

bit 1 WUE: Wake-up Enable bit

Asynchronous mode:

1 = EUSART will continue to sample the RX pin – interrupt generated on falling edge; bit cleared in hardware on following rising edge

0 = RX pin not monitored or rising edge detected

Synchronous mode:

Unused in this mode.

bit 0 ABDEN: Auto-Baud Detect Enable bit

Asynchronous mode:

1 = Enable baud rate measurement on the next character. Requires reception of a Sync field (55h); cleared in hardware upon completion.

0 = Baud rate measurement disabled or completed

Synchronous mode:

Unused in this mode.

Generador de relación de Baudio (BRG)

El PIC18F4550 dispone de un generador de velocidad de comunicación. Se trata de un temporizador cuya frecuencia se controla mediante un registro que puede ser de 8 (SPBRG) o 16 bits ([SPBRGH;SPBRG]).

El generador de velocidad de comunicación tiene dos velocidades de funcionamiento que pueden seleccionarse mediante el bit BRGH (TXSTA). En la siguiente tabla se establece como calcular la velocidad de comunicación para el modo asíncrono en función de los valores de los bits BRGH y BRG16 y de los registros SPBRGH y SPBRG

20.1 Baud Rate Generator (BRG)

The BRG is a dedicated 8-bit, or 16-bit, generator that supports both the Asynchronous and Synchronous modes of the EUSART. By default, the BRG operates in 8-bit mode. Setting the BRG16 bit (BAUDCON<3>) selects 16-bit mode.

The SPBRGH:SPBRG register pair controls the period of a free-running timer. In Asynchronous mode, bits, BRGH (TXSTA<2>) and BRG16 (BAUDCON<3>), also control the baud rate. In Synchronous mode, BRGH is ignored. Table 20-1 shows the formula for computation of the baud rate for different EUSART modes which only apply in Master mode (internally generated clock).

Given the desired baud rate and F_{osc} , the nearest integer value for the SPBRGH:SPBRG registers can be calculated using the formulas in Table 20-1. From this, the error in baud rate can be determined. An example calculation is shown in Example 20-1. Typical baud rates and error values for the various Asynchronous modes are shown in Table 20-2. It may be advantageous

Ejemplo 1:

TXSTA<4> SYNC = 0 (asíncrono)

TXSTA<2> BRGH=1 (High speed)

BAUDCON<3> BRG16= 0 (8bit)

Donde: n es el valor en SPBRG (0 a 255)

$$\text{Baudio} = \frac{F_{osc}}{[16 (n + 1)]}$$

Ejemplo 2:

TXSTA<4> SYNC = 0 (asíncrono)

TXSTA<2> BRGH=1 (High speed)

BAUDCON<3> BRG16= 1 (16bit)

Donde: n es el valor en SPBRGH:SPBRG (0 a 65535)

$$\text{Baudio} = \frac{F_{osc}}{[4 (n + 1)]}$$

TABLE 20-1: BAUD RATE FORMULAS

Configuration Bits			BRG/EUSART Mode	Baud Rate Formula
SYNC	BRG16	BRGH		
0	0	0	8-bit/Asynchronous	$F_{osc}/[64 (n + 1)]$
0	0	1	8-bit/Asynchronous	$F_{osc}/[16 (n + 1)]$
0	1	0	16-bit/Asynchronous	$F_{osc}/[4 (n + 1)]$
0	1	1	16-bit/Asynchronous	
1	0	x	8-bit/Synchronous	
1	1	x	16-bit/Synchronous	

Legend: x = Don't care, n = value of SPBRGH:SPBRG register pair

Generador de relación de Baudio (BRG)

15

Ejemplo: Calcule el valor del registro SPBRG para una velocidad de 9600 baudio, $F_{osc}=16\text{MHz}$ a baja velocidad ($\text{BRGH}=0$) y 8Bit ($\text{BRG16}=0$)

Solución:

$$\text{Baudio} = \frac{F_{osc}}{64(n+1)}$$

$$9600 = \frac{16\text{MHz}}{64(n+1)}$$

$$n = \frac{16\text{MHz}}{9600 * 64} - 1$$

$$n = 25,0416 \approx 25$$

El registro SPBRG se debe cargar con .25

Calculo del error:

$$\%Error = \frac{\text{Valor calculado} - \text{Valor deorado}}{\text{Valor deorado}} * 100$$

$$\text{Valor calculado} = \frac{16\text{MHz}}{64(25+1)} = 9615,384615$$

$$\%Error = \frac{9615,384615 - 9600}{9600} * 100 = 0,16\%$$

Generador de relación de Baudio (BRG)

16

Ejemplo: Calcule el valor del registro SPBRG para una velocidad de 9600 baudio, Fosc=16MHz a baja velocidad (BRGH=0) y 8Bit (BRG16=0)

BAUD RATE (K)	Fosc = 20 MHz			Fosc = 16 MHz			Fosc = 10 MHz		
	KBAUD	% ERROR	SPBRG value (decimal)	KBAUD	% ERROR	SPBRG value (decimal)	KBAUD	% ERROR	SPBRG value (decimal)
0.3	-	-	-	-	-	-	-	-	-
1.2	1.221	1.75	255	1.202	0.17	207	1.202	0.17	129
2.4	2.404	0.17	129	2.404	0.17	103	2.404	0.17	64
9.6	9.766	1.73	31	9.615	0.16	25	9.766	1.73	15
19.2	19.531	1.72	15	19.231	0.16	12	19.531	1.72	7
28.8	31.250	8.51	9	27.778	3.55	8	31.250	8.51	4
33.6	34.722	3.34	8	35.714	6.29	6	31.250	6.99	4
57.6	62.500	8.51	4	62.500	8.51	3	52.083	9.58	2
HIGH	1.221	-	255	0.977	-	255	0.610	-	255
LOW	312.500	-	0	250.000	-	0	156.250	-	0

Generador de relación de Baudio (BRG)

17

Ejercicio:

Para una velocidad de 9600 baudio

Frecuencia de 20Mhz

TXSTA<4> SYNC = 0 (asíncrono)

TXSTA<2> BRGH=1 (High speed)

BAUDCON<3> BRG16= 0 (8bit)

SPBRG= ?

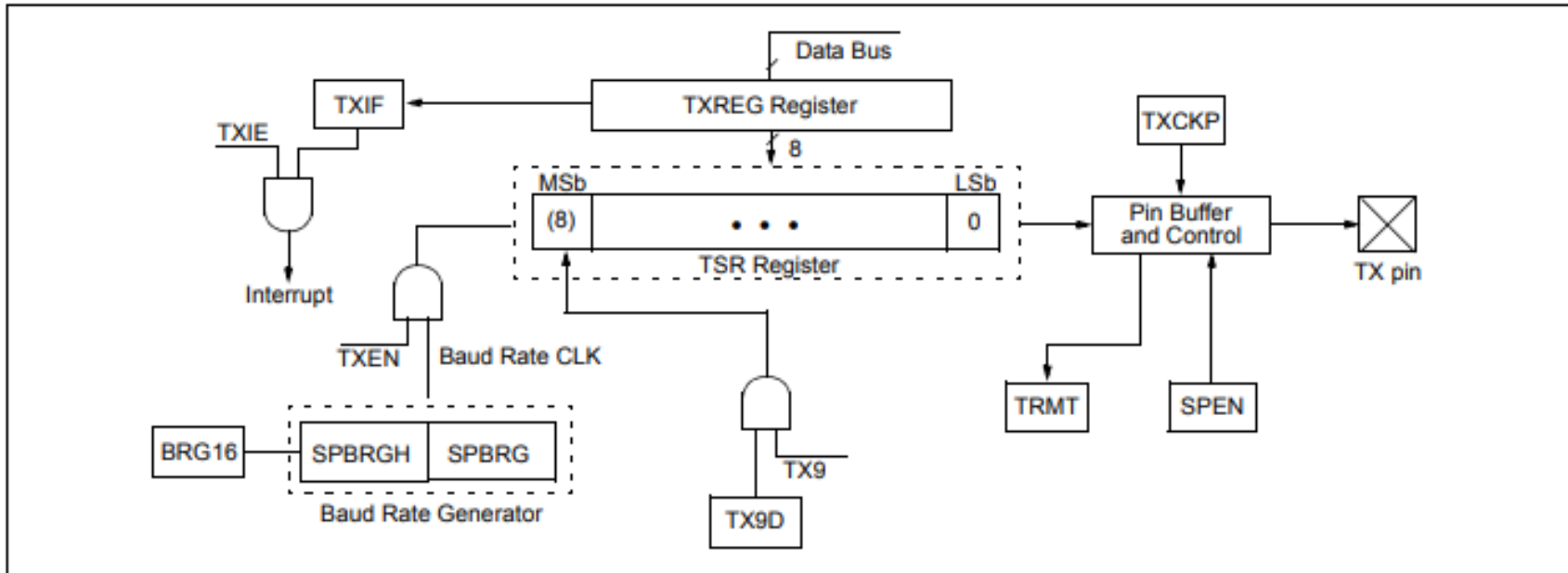
$$\text{Baudio} = \frac{F_{osc}}{[16 (n + 1)]}$$

Solución:

BAUD RATE (K)	SYNC = 0, BRGH = 1, BRG16 = 0											
	Fosc = 40.000 MHz			Fosc = 20.000 MHz			Fosc = 10.000 MHz			Fosc = 8.000 MHz		
	Actual Rate (K)	% Error	SPBRG value (decimal)	Actual Rate (K)	% Error	SPBRG value (decimal)	Actual Rate (K)	% Error	SPBRG value (decimal)	Actual Rate (K)	% Error	SPBRG value (decimal)
0.3	—	—	—	—	—	—	—	—	—	—	—	—
1.2	—	—	—	—	—	—	—	—	—	—	—	—
2.4	—	—	—	—	—	—	2.441	1.73	255	2.403	-0.16	207
9.6	9.766	1.73	255	9.615	0.16	129	9.615	0.16	64	9.615	-0.16	51
19.2	19.231	0.16	129	19.231	0.16	64	19.531	1.73	31	19.230	-0.16	25
57.6	58.140	0.94	42	56.818	-1.36	21	56.818	-1.36	10	55.555	3.55	8
115.2	113.636	-1.36	21	113.636	-1.36	10	125.000	8.51	4	—	—	—

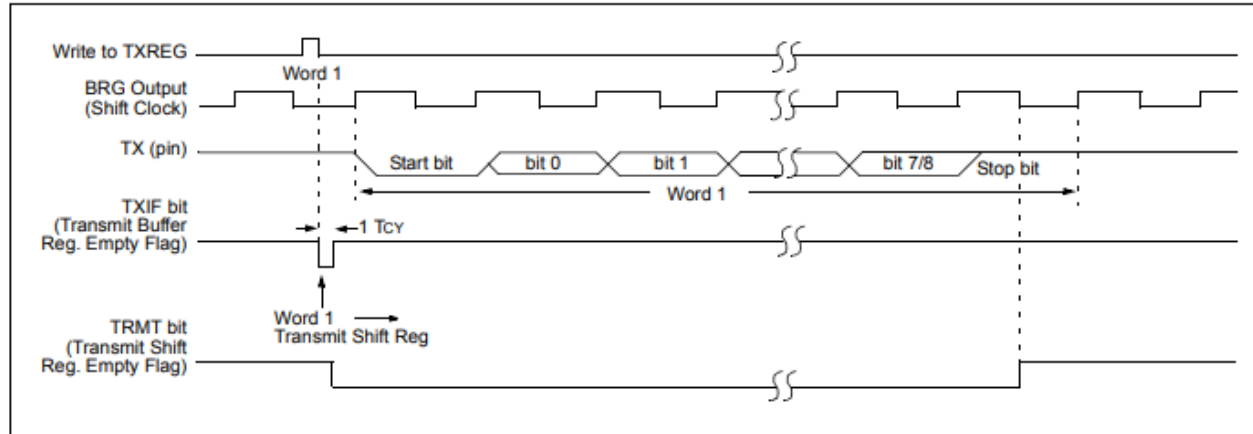
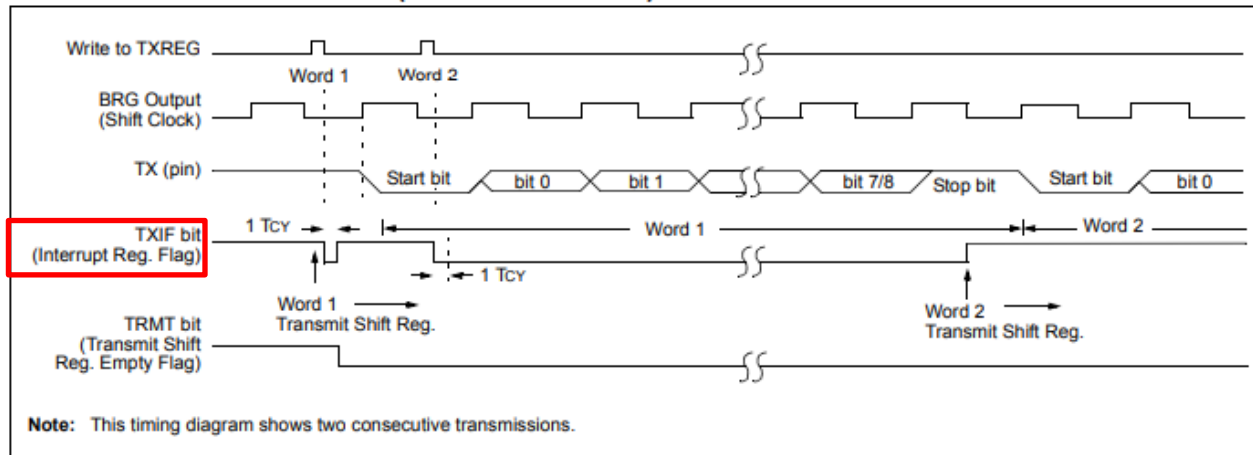
EUSART - Diagrama de bloque para la transmisión

18

FIGURE 20-3: EUSART TRANSMIT BLOCK DIAGRAM

EUSART - Diagrama de bloque para la transmisión

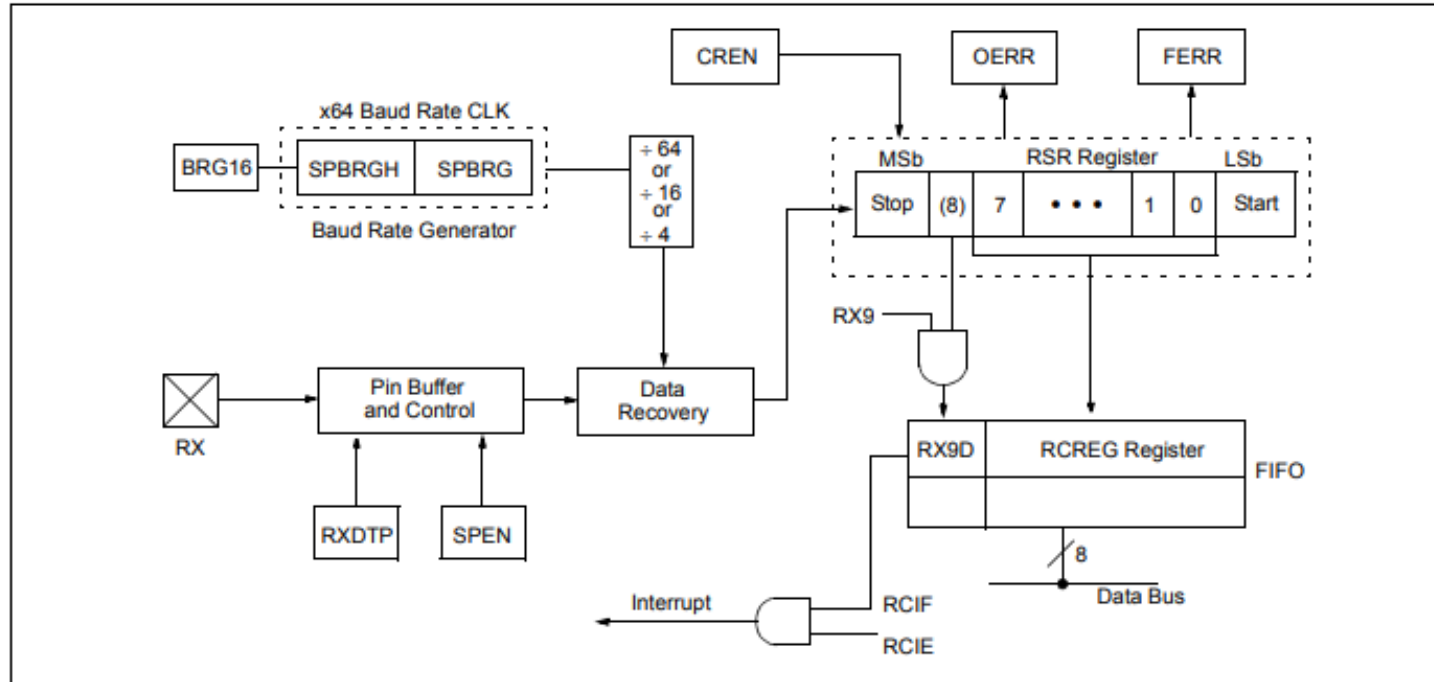
19

FIGURE 20-4: ASYNCHRONOUS TRANSMISSION, TXCKP = 0 (TX NOT INVERTED)**FIGURE 20-5: ASYNCHRONOUS TRANSMISSION (BACK TO BACK), TXCKP = 0 (TX NOT INVERTED)**

EUSART - Diagrama de bloque para la recepción

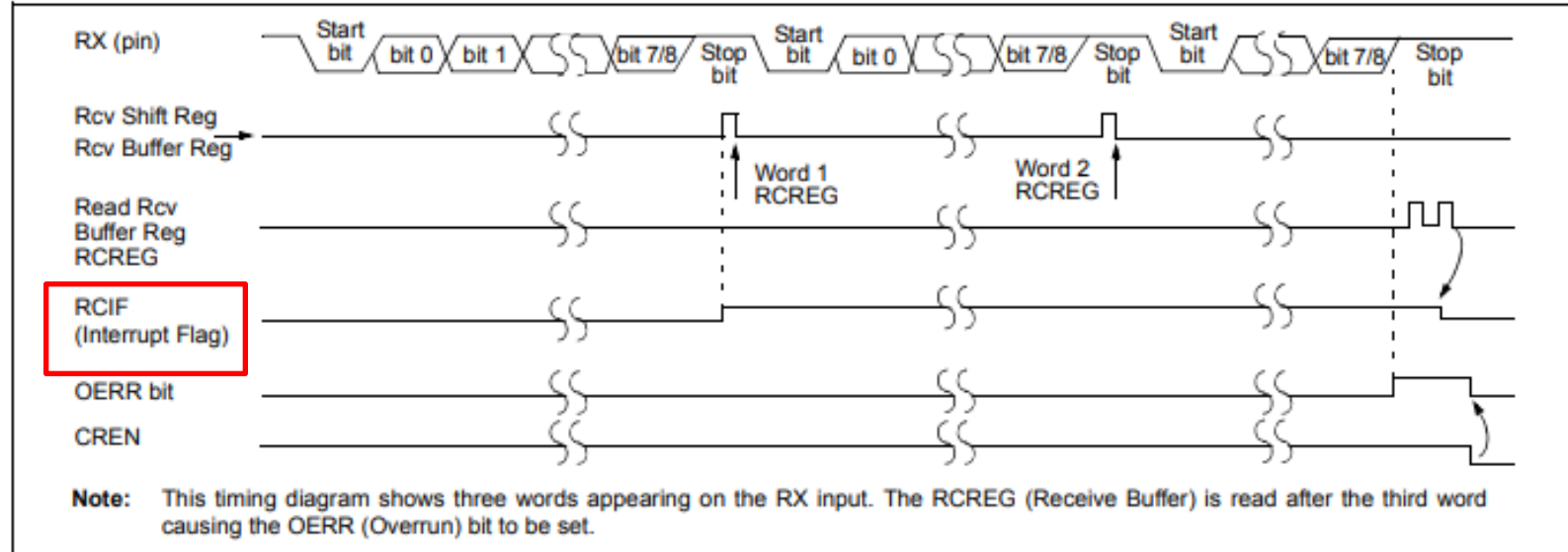
20

FIGURE 20-6: EUSART RECEIVE BLOCK DIAGRAM



EUSART - Diagrama de bloque para la recepción

21

FIGURE 20-7: ASYNCHRONOUS RECEPTION, RXDTP = 0 (RX NOT INVERTED)

Pasos para preparar una transmisión serie asíncrona

22

To set up an Asynchronous Transmission:

1. Initialize the SPBRGH:SPBRG registers for the appropriate baud rate. Set or clear the BRGH and BRG16 bits, as required, to achieve the desired baud rate.
2. Enable the asynchronous serial port by clearing bit, SYNC, and setting bit, SPEN.
3. If the signal from the TX pin is to be inverted, set the TXCKP bit.
4. If interrupts are desired, set enable bit, TXIE.
5. If 9-bit transmission is desired, set transmit bit, TX9. Can be used as address/data bit.
6. Enable the transmission by setting bit, TXEN, which will also set bit, TXIF.
7. If 9-bit transmission is selected, the ninth bit should be loaded in bit, TX9D.
8. Load data to the TXREG register (starts transmission).
9. If using interrupts, ensure that the GIE and PEIE bits in the INTCON register (INTCON<7:6>) are set.

Pasos para preparar una recepción serie asíncrona

23

To set up an Asynchronous Reception:

1. Initialize the SPBRGH:SPBRG registers for the appropriate baud rate. Set or clear the BRGH and BRG16 bits, as required, to achieve the desired baud rate.
2. Enable the asynchronous serial port by clearing bit, SYNC, and setting bit, SPEN.
3. If the signal at the RX pin is to be inverted, set the RXDTP bit.
4. If interrupts are desired, set enable bit, RCIE.
5. If 9-bit reception is desired, set bit, RX9.
6. Enable the reception by setting bit, CREN.
7. Flag bit, RCIF, will be set when reception is complete and an interrupt will be generated if enable bit, RCIE, was set.
8. Read the RSTA register to get the 9th bit (if enabled) and determine if any error occurred during reception.
9. Read the 8-bit received data by reading the RCREG register.
10. If any error occurred, clear the error by clearing enable bit, CREN.
11. If using interrupts, ensure that the GIE and PEIE bits in the INTCON register (INTCON<7:6>) are set.

Pasos para configurar Modulo SCI en modo asíncrono (full dúplex)

1. Configurar puertos, en el registro TRISC
 - RC7 Como entrada
 - RC6 Como salida
2. Cargar valor en SPBRG (para 8bit)
3. Configurar el registro TXSTA
 - Seleccionar tamaño bit de datos de transmisión de 8 bit.(TX9=0)
 - habilitar transmisión . (TXEN=1)
 - Definirlo como Asíncrono. (SYNC=0)
 - Seleccionar velocidad de transmisión alta (BRGH=1)
4. Configurar el registro RCSTA
 - Habilitar el modulo SCI (SPEN=1)
 - Seleccionar tamaño bit de datos de resección de 8 (RX9=0)
 - activar resección continua (CREN=1)
5. Configurar registro BAUDCON
 - Definir polaridad de transmisión y recepción como no invertido (RXDTP=0 y TXDTP=0)
 - Seleccionar generador de baudio de 8bit (BRG16=0)
 - Deshabilitar el modo de auto-detección de velocidad de comunicación (ABDEN=0)
6. Habilitar interrupción (opcional)
7. para iniciar una transmisión
 - Cargar el 9bit si es necesario
 - Cargar el dato en TXREG (se inicia la transmisión)
8. Para obtener el dato recibido
 - Al completarse la resección de un dato: RCIF=1, y si se habilitaron las interrupciones se generara una.
 - Leer el dado en RCREG.
 - leer el 9bit si es necesario

EUSART - Rutinas

25

CONFIGURAR_TRANSMISOR_RECEPTOR

```
;BSF      TRISC,7
;BCF      TRISC,6

MOVLW     D'129'
MOVWF     SPBRG

MOVLW     B'00100100'
MOVWF     TXSTA

MOVLW     B'10010000'
MOVWF     RCSTA

MOVLW     B'00000000'
MOVWF     BAUDCON

RETURN
```

RECIBIR_DATO

```
BTFSS     PIR1,RCIF
GOTO      RECIBIR_DATO
MOVF      RCREG,W
MOVWF     DATO_RX
BCF       PIR1,RCIF
RETURN
```

ENVIAR_DATO

```
BTFSS     PIR1,TXIF
GOTO      ENVIAR_DATO
MOVWF     TXREG
CALL      RETARDO_5MS
RETURN
```

VERIFICAR_ERROR_DE_OVERRUN

```
BTFSS     RCSTA,OERR
RETURN

BCF       RCSTA,CREN
BCF       RCSTA,OERR
BSF       RCSTA,CREN
RETURN
```

Ejemplo:

Escriba un programa para comunicar dos microcontroladores usando el modulo SCI. Colocar una pantalla LCD y un teclado.
Realice: A) los cálculos, B) el diagrama de conexión, y C) el código en lenguaje ensamblador.

Solución: A

Para una velocidad de 9600 baudio, Fosc=20MHz, alta velocidad (BRGH=1) y 8Bit (BRG16=0)

$$9600 = \frac{20Mhz}{[16 (n + 1)]} \Rightarrow n = \frac{20Mh}{[16 * 9600]} - 1$$

$$n = 129,2$$

Entonces SPBRG=.129

Calculo del error:

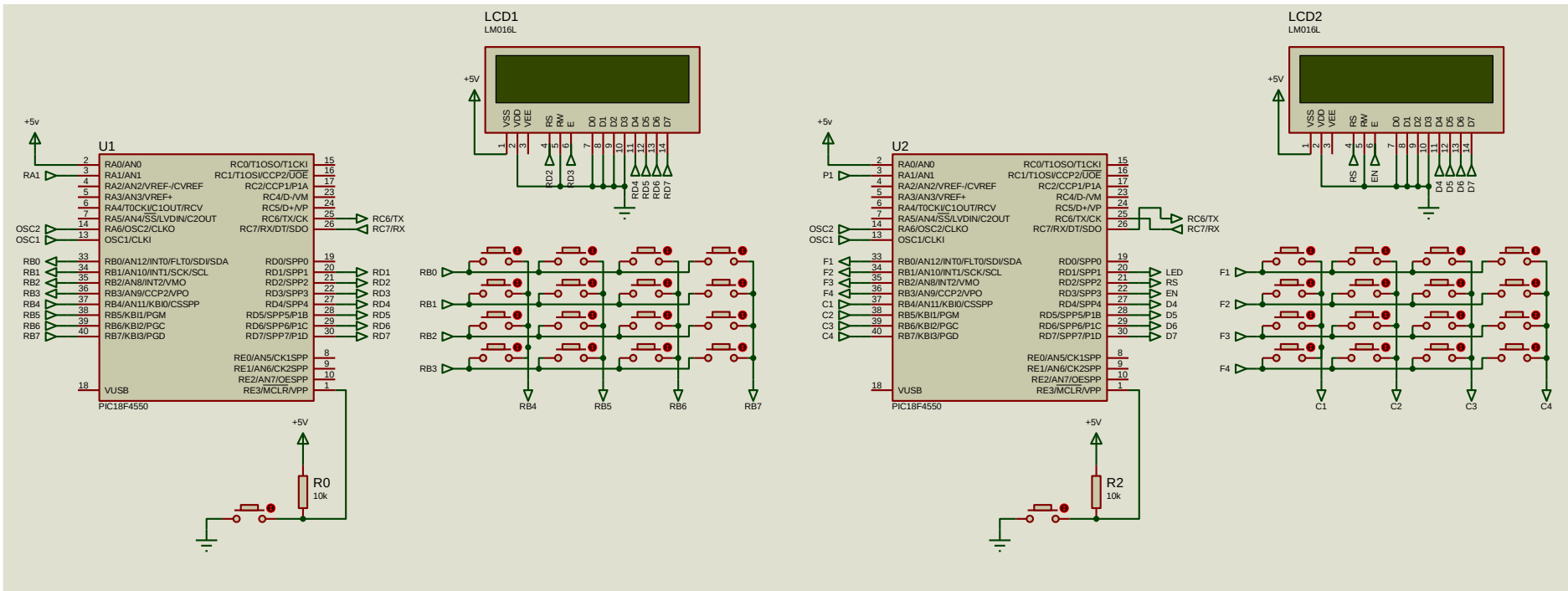
$$Valor\ calculado = \frac{20MHz}{16(129 + 1)} = 9615,384615$$

$$\%Error = \frac{9615,384615 - 9600}{9600} * 100 = 0,16\%$$

Ejemplo:

Escriba un programa para comunicar dos microcontroladores usando el modulo SCI. Colocar una pantalla LCD y un teclado.

Realice: A) los cálculos, B) el diagrama de conexión, y C) el código en lenguaje ensamblador.

Solución: B

Ejemplo:

Escriba un programa para comunicar dos microcontroladores usando el modulo SCI. Colocar una pantalla LCD y un teclado.

Realice: A) los cálculos, B) el diagrama de conexión, y C) el código en lenguaje ensamblador.

Solución: C

```
INICIO
    CALL CONF_PUERTOS
    CALL CONF_LCD
    CALL CONF_USART
    CALL CONF_INT ; teclado y RX
    GOTO PROGRAMA
```

```
ATENDER_INTERRUPCION_H
    BCF     INTCON,GIE
    MOVWF   W_TEMP
    MOVFF   STATUS,STATUS_TEMP
    MOVFF   BSR,BSR_TEMP

    BTFSC   INTCON,RBIF
    GOTO    INTERRUPCION_RB4_RB7
    BTFSC   PIR1,RCIF
    GOTO    INTERRUPCION_RX
```

```
SALIR_INTERRUPCION
    MOVFF   BSR_TEMP,BSR
    MOVF    W_TEMP,W

    MOVFF   STATUS_TEMP,STATUS
    BSF     INTCON,GIE
    RETFIE
```

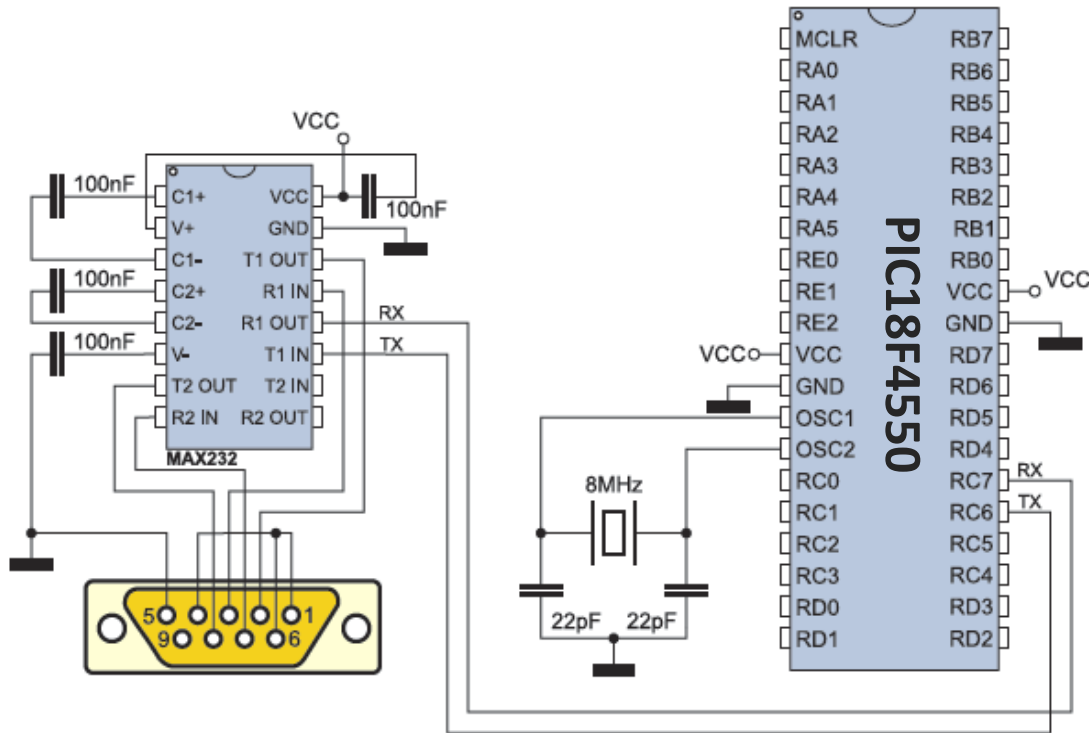
```
INTERRUPCION_RB4_RB7
    BCF     INTCON,RBIF
    CALL    TECLA_PRESIONADA
    MOVF    NTECLA,F
    BZ      SALIR_INTERRUPCION
    CALL    TABLA_TECLADO_REAL_ASCII
    MOVWF   DATO_TX
    CALL    LCD_DATO
    MOVF    DATO_TX,W
    CALL    ENVIAR_DATO_USART
    GOTO    SALIR_INTERRUPCION
;=====
INTERRUPCION_RX
    BCF     PIR1,RCIF
    CALL    VERIFICAR_ERROR_DE_OVERRUN
    MOVF    RCREG,W
    CALL    LCD_DATO
    GOTO    SALIR_INTERRUPCION
```

PROGRAMA

```
                                GOTO PROGRAMA
;=====
    INCLUDE RETARDOS_20MHZ.INC
    INCLUDE LCD.INC
    INCLUDE TECLADO_MATRICIAL.INC
    INCLUDE LIBRERIA_USART.INC
;=====
    END
```

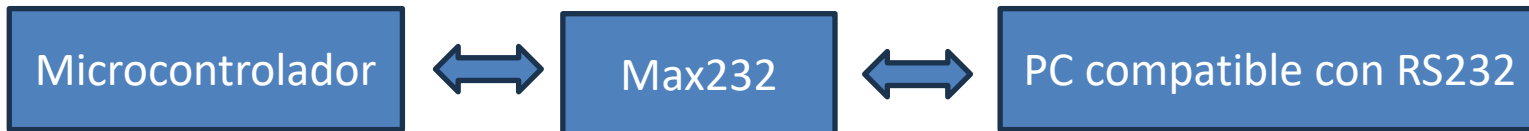
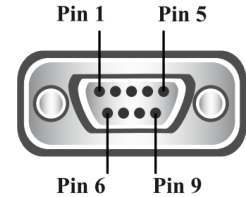
Conexión de transmisión y recepción asíncrona en modo full-duplex el microcontrolador y un pc

29



Pin 1	DCD
Pin 2	RXD
Pin 3	TXD
Pin 4	DTR
Pin 5	GND
Pin 6	DSR
Pin 7	RTS
Pin 8	CTS
Pin 9	RI

RS232 Pinout (9 Pin Male)



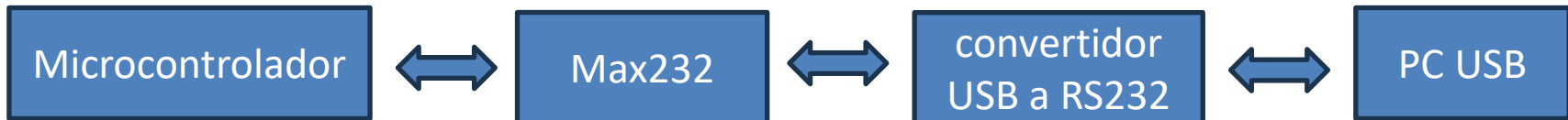
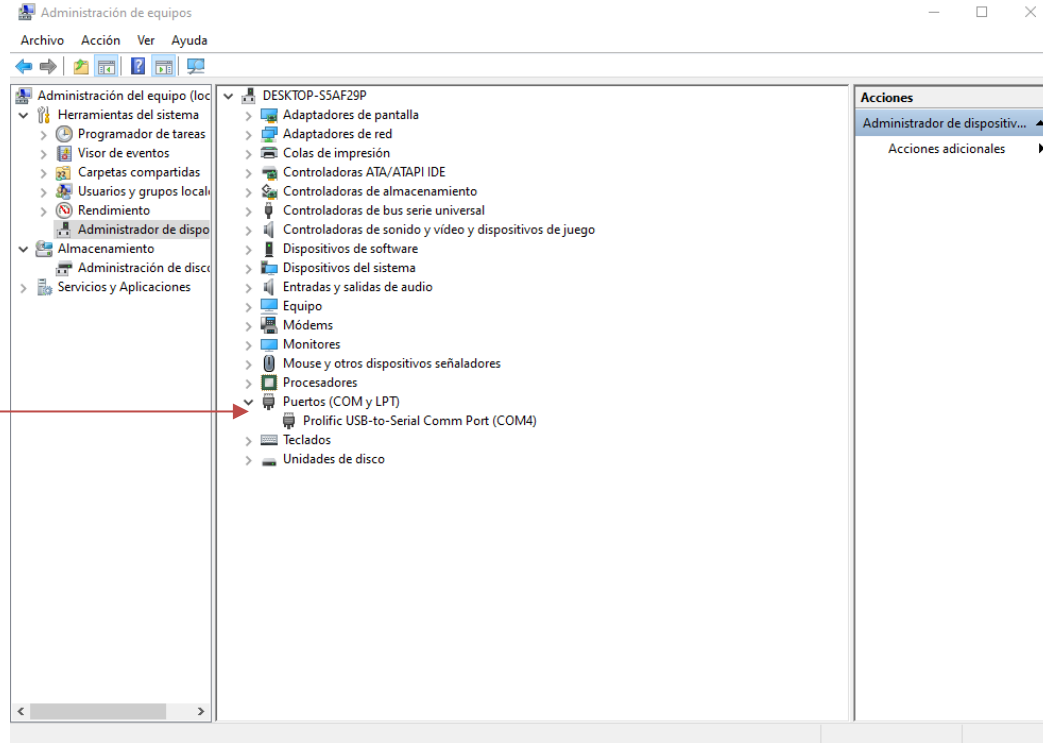
Convertidor USB a RS232

30

Un convertidor USB a RS232 permite la comunicación entre un puerto USB de un ordenador y un dispositivo con interfaz RS232.



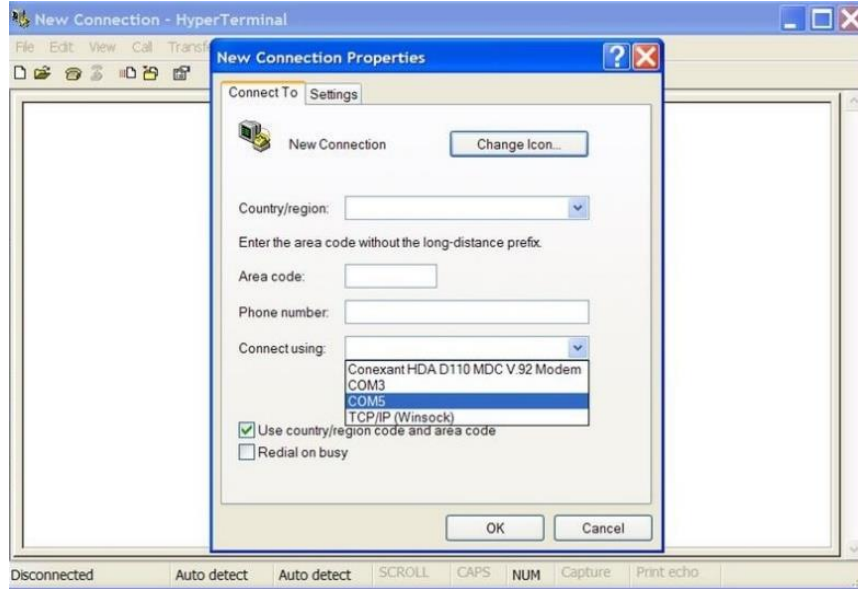
[Descargar Driver. PL2303](#)



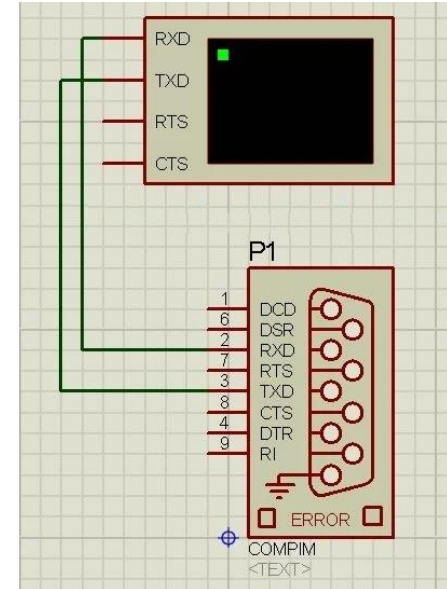
Comunicación con un Terminal

31

HyperTerminal



Proteus



Microcontrolador
(físico)



PC
(terminal o
proteus)

No se requiere usar un COM virtual

Microcontrolador
(en proteus)



PC
(terminal o
proteus)

se requiere usar un COM virtual